Hydro AI: how the project works

A walkthrough of the phone app, screen by screen, for the team. Maize in a hydroponic rig: one ESP32 with sensors and relays, one Android phone. No laptop, router or internet needed.

The big picture

Sensors
DHT11 air temp + humidity, HC-SR04 water level, soil probe (pH / TDS later)

→

ESP32 board
Reads every 10 s, keeps ~3 h of readings, switches the relays

⇄

Phone app
Stores readings, works out today's targets, decides fan / light / pump / mister

→

Relays
Fan, grow light, refill pump, mister (heater + dosing pumps optional)

Everything "smart" runs **on the phone**. Every few seconds the app pulls the board's new readings, saves them in a local database, checks them against the maize targets for the current growth stage, and sends relay commands back. It also tells the board what targets to hold by itself.

When the app is closed or the phone walks away, the board keeps the plant on those last targets with simple on/off rules. It never doses and never pumps blind. It also buffers readings, and the app pulls in that backlog next time it connects, so the history has no gaps.

What's on the phone

Part	What it does
Stage targets	Germination → grain fill targets for maize (air temp, humidity, light hours), from `config/maize.json`
Optimizer (ML)	Learns daily water uptake vs. conditions and nudges the targets, but only when it can actually predict unseen days
Controller	Hysteresis rules that turn targets into relay on/off, so relays don't chatter
Alerts	Air outside the stage range, tank running dry, sensor faults, board out of reach (no contact for 90 s), leaf problems → phone notifications
Leaf check	Leaf-colour analysis + an on-device disease model (common rust, northern leaf blight, gray leaf spot)
Ask (advisor)	Offline crop advisor: describe symptoms in English, Hindi or Hinglish, get likely causes and a treatment plan

Connecting phone and board

There are two ways. The app tries both by itself, so you don't need to change any settings:

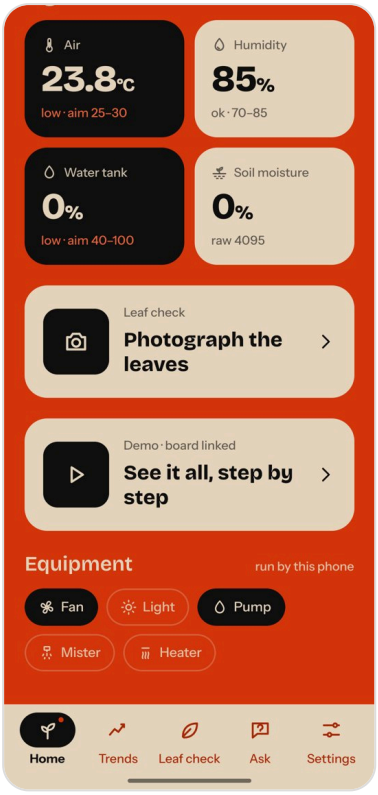
1. **Phone hotspot (what we use).** The board joins the phone's hotspot (set under *Settings → Board → Router or hotspot*). The phone can stay on mobile data or campus WiFi as normal. The app searches the hotspot for the board and remembers where it found it.
2. **Board's own WiFi.** Join `HydroAI-maize` in the phone's WiFi settings. Android will say "no internet"; choose to stay connected. The board is always at `192.168.4.1` there.

> **If it says "Offline":** make sure the hotspot is on, then power-cycle the board. After a restart it joins the hotspot within a few seconds, and the app links on its next check. Turn off *Automatically turn off personal hotspot* in the phone's hotspot settings. Otherwise the phone switches the hotspot off whenever the board drops off for a while.

1 · Home: how is the plant doing?



Status + live readings



Shortcuts + equipment

Top bar

Crop, day of the grow and growth stage (*Maize · day 1 · Germination*). The pill on the right says **Live** when the board is answering, or **Offline · N min ago** when it isn't.

Your plant

A one-line verdict ("Looks good" / "Needs you now") with the reasons underneath, e.g. reservoir empty or air too cold for this stage. The bar shows how far through the ~111-day grow we are.

Right now

The latest reading from each sensor next to its target band for today's stage. **low / ok / high** tells you at a glance which one is off.

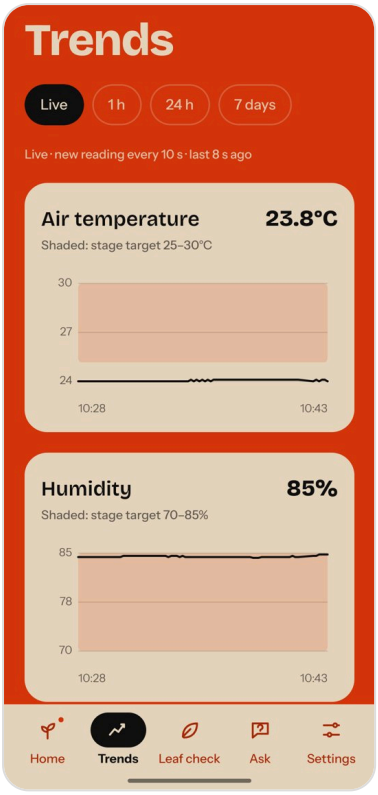
Equipment

Which relays are on right now (filled chips = on). The phone decides these; you don't toggle them by hand in normal use.

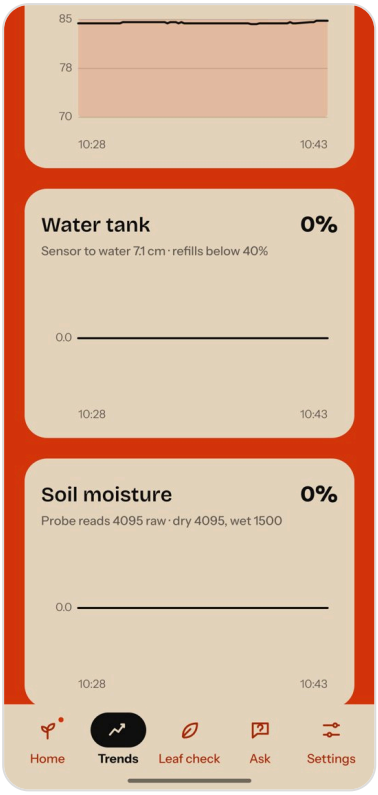
Shortcuts

Photograph the leaves opens Leaf check. *See it all, step by step* opens the guided demo (section 6).

2 · Trends: history and targets



Air temp + humidity



Water tank + soil

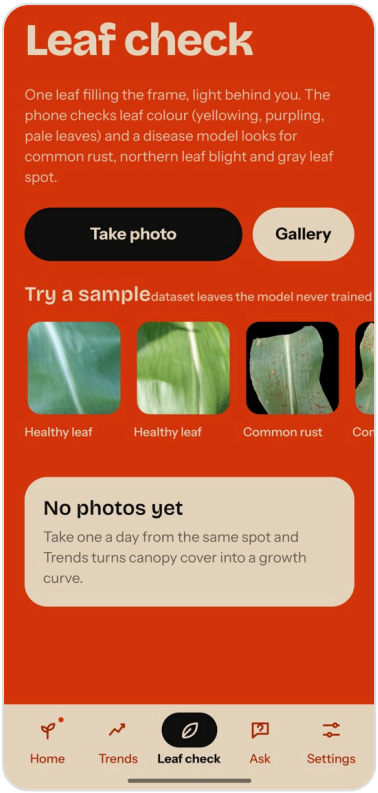
One chart per sensor, over **Live**, **1 h**, **24 h** or **7 days**. The shaded band is the target range for the current growth stage, so it's easy to see how long conditions spent outside it.

Press and drag on a chart to read exact values.

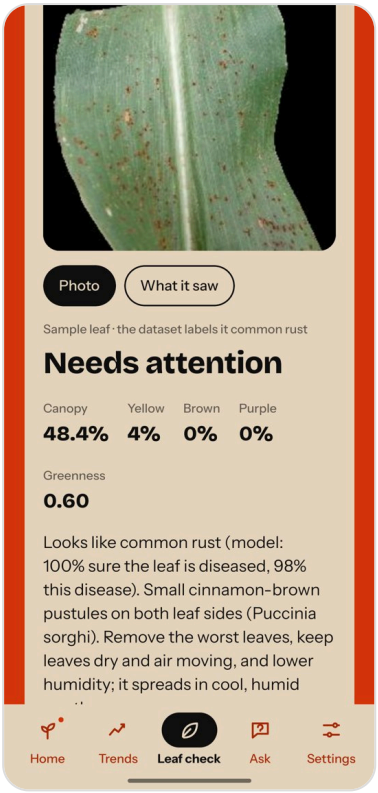
The water-tank chart matters most for the ML part. The **drop in water level between refills** is how the app measures how much the plant drinks each day. That daily uptake is the growth signal the optimizer learns from.

> The tank % depends on calibration (Settings → Calibration → Water tank: the sensor-to-water distance when the tank is empty and when it's full). If it reads 0% with water in the tank, re-do that calibration.

3 · Leaf check: photo diagnosis



Take a photo or try a sample



Result for a rust sample

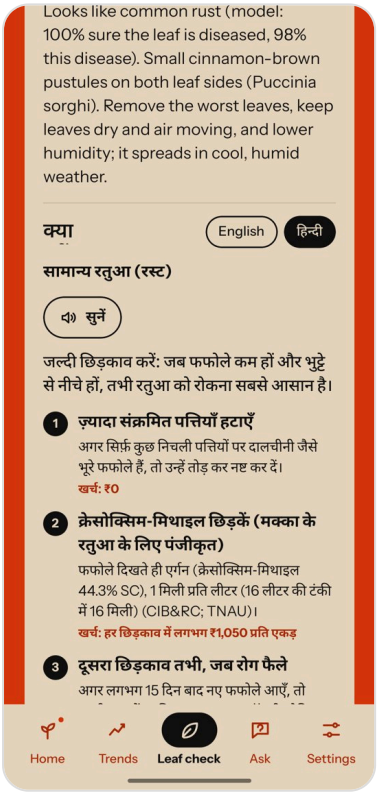
Take one leaf filling the frame with light behind you, or pick a photo from the gallery. **Try a sample** runs dataset leaves the model never saw in training, which is handy for demos.

What it measures

- **Colour analysis:** canopy cover, yellow / brown / purple share and greenness. Yellowing and purpling point to nutrient problems.
- **Disease model** (on-device): healthy vs. diseased, then which disease (common rust, northern leaf blight, gray leaf spot), with its confidence.

What it saw shows the mask the analysis used, so you can tell if it picked up the background instead of the leaf.

Taking one photo a day from the same spot turns canopy cover into a growth curve on Trends.

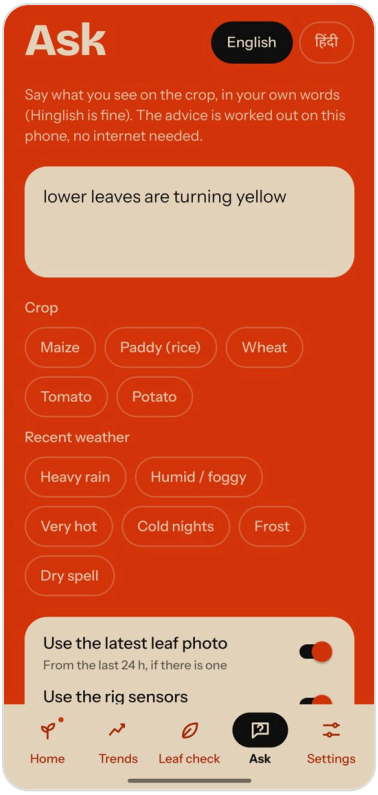


Treatment plan (Hindi shown)

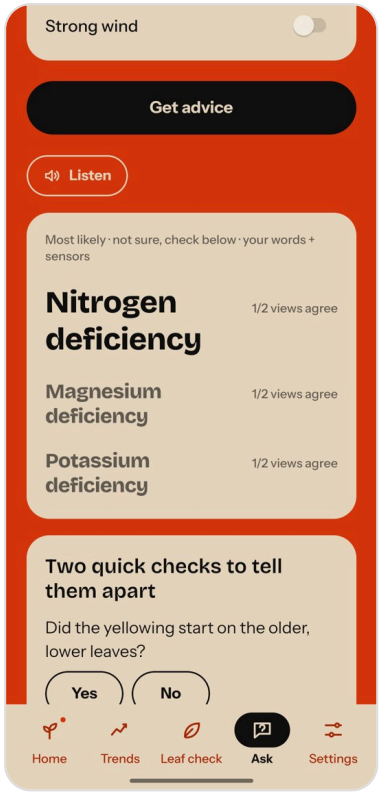
Treatment plan

For a disease, the app gives numbered steps with what to spray, the dose per litre and an approximate cost per acre. Toggle **English / हिंदी**; **Listen** reads it aloud.

4 · Ask: the offline crop advisor



Describe the symptom



Likely causes + follow-up

Type what you see in plain words, e.g. *"lower leaves are turning yellow"* or Hinglish *"makka ke patte peele ho rahe hain"*. Pick the crop and any recent weather.

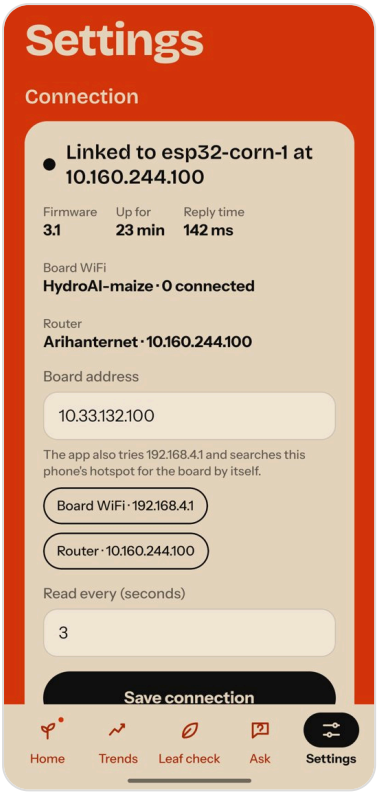
Two switches add evidence automatically: **Use the latest leaf photo** (from Leaf check, last 24 h) and **Use the rig sensors** (live readings from the board).

The answer

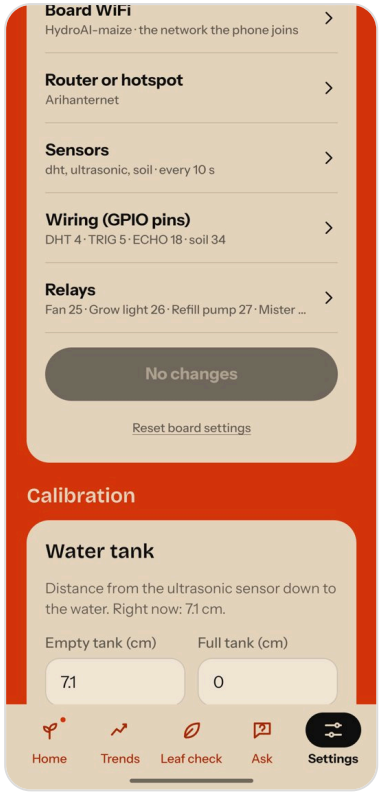
A ranked list of likely causes, with how many independent "views" (your words, the photo, the sensors) agree on each. When two causes look alike, it asks **quick yes/no checks** to tell them apart, e.g. "Did the yellowing start on the older, lower leaves?" That separates nitrogen from magnesium deficiency.

All of this runs on the phone with no internet. **Listen** reads the advice aloud.

5 · Settings: connection and the board



Connection status



Board + calibration

Connection

Shows the board's ID and the address it's linked at, firmware version, uptime and reply time. **Board WiFi** is the board's own network (and how many phones are on it). **Router** is the hotspot/router it joined and its address there.

Board address is optional: the app also tries `192.168.4.1` and searches the phone's hotspot by itself. **Blink the board LED** is a quick way to check you're talking to the right board.

Board

Settings saved in the ESP32's memory, so **changing hardware never needs a reflash**:

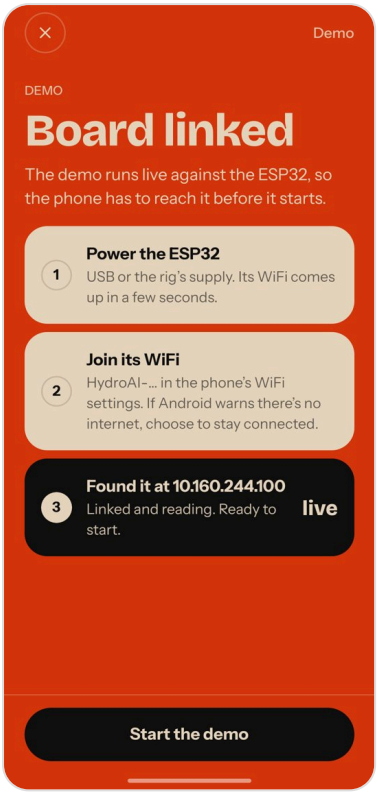
- Board WiFi name/password, and the router or hotspot it should join
- Which sensors are fitted and the reading interval
- Wiring: GPIO pin for each sensor and relay. Wrong or clashing pins are refused before saving.

Saving restarts the board (about 10 s).

Calibration

Water-tank empty/full distances, so the ultrasonic reading becomes a %. Also the refill thresholds.

6 · Guided demo: for judges and visitors



Demo start screen

Home → *See it all, step by step*. It first checks the board is linked (step 3 turns black with **live** and the address it found). Then **Start the demo** walks through the system on the real hardware: each sensor and its live value, how the ultrasonic distance becomes tank level and litres, today's targets for the growth stage, a relay test you drive by hand, one full control cycle (ask the board → store → work out targets → decide → command), and the alerts the app watches for.

During the relay test, automatic control is paused so the demo's commands aren't overwritten. It resumes when the demo ends.

Before a demo, check

1. Phone hotspot on, with auto-turn-off disabled
2. Board powered; restart it if Home says Offline
3. Home shows **Live** and readings say "updated a few s ago"
4. Water tank calibrated, so it doesn't open on "Reservoir at 0%"

Where things live in the repo (`esa_project/`)

Folder	What
`mobile/`	The Android app (Expo / React Native). `src/engine` = ML, controller, advisor; `src/lib` = board link, database, alerts
`firmware/`	ESP32 code (PlatformIO): `pio run -t upload`
`hydro/`, `advisor/`	The original Python versions; the app's TypeScript is tested against them
`config/maize.json`	Stage-by-stage biology targets (biology team: please verify)
`simulator/`	Fake ESP32 / simulated grow for testing without hardware
`report/`	Full write-up of the ML approach and experiments